

Testanhang / Testprotokoll

Zweck des Testanhangs

Der Testanhang dokumentiert die durchgeführten technischen Prüfungen der Bildungsplan-App. Er dient als Nachweis, dass zentrale Funktionen der Angular-Anwendung automatisiert getestet, die Testresultate ausgewertet und die Ergebnisse nachvollziehbar festgehalten wurden. Der Fokus liegt auf Unit-Tests mit Vitest, der Prüfung der Testabdeckung, der Build-Kontrolle sowie dem Nachweis der Versionierung über Git.

Die Testdokumentation unterstützt die Nachvollziehbarkeit der Kontrollphase. Die Kriterien verlangen eine beschriebene Testumgebung, reproduzierbare Testszenarien, erwartete Ergebnisse und übersichtlich dokumentierte Resultate.

Testumgebung

Bereich	Angabe
Projekt	Bildungsplan-Frontend
Framework	Angular 20
Sprache	TypeScript
Testframework	Vitest
Testausführung	Angular Unit-Test-Builder
Entwicklungsumgebung	Lokale Entwicklungsumgebung unter Windows
Repository	Git / GitLab
Backend	Bestehende REST-API, lokal gestartet
Testbefehle	npm test, npm run test:coverage, npm run build

Reproduzierbarkeit

Die Tests können durch eine externe Fachperson reproduziert werden, indem im Projektverzeichnis die Abhängigkeiten installiert und danach die Testbefehle ausgeführt werden.

```
npm install
npm test
npm run test:coverage
npm run build
```

Für die manuelle Prüfung muss zusätzlich das bestehende Backend lokal gestartet werden. Danach kann das Frontend gestartet und mit der lokalen API verwendet werden.

Zusammenfassung der automatisierten Testergebnisse

Prüfung	Ergebnis
Testdateien	17 bestanden
Tests	19 bestanden
Teststatus	Erfolgreich
Testdauer	12.24 Sekunden
Coverage Statements	82.22 %
Coverage Branches	72.4 %
Coverage Functions	72.5 %
Coverage Lines	82.22 %
Build-Prüfung	Erfolgreich
Initial Bundle Total	3.34 MB

Die Unit-Tests wurden erfolgreich ausgeführt. Alle 17 Testdateien und alle 19 Tests wurden bestanden. Zusätzlich wurde ein Coverage-Bericht erstellt. Die Line-Coverage und Statement-Coverage betragen jeweils 82.22 %. Damit wurde das angestrebte Coverage-Ziel erreicht.

Testprotokoll

Testfall	Testziel	Vorgehen	Erwartetes Ergebnis	Effektives Ergebnis	Status
T-01	State-Logik prüfen	Unit-Tests für den BildungsplanStateService ausführen. Geprüft werden EFZ-Auswahl, Fachrichtung, Suchbegriff, Lernort, Lehrjahr, Modultyp, Kontextprüfung und Zurücksetzen des Zustands.	Der State speichert Werte korrekt, setzt abhängige Zustände zurück und erkennt den vollständigen Kontext korrekt.	Die Tests wurden erfolgreich ausgeführt. Die State-Logik funktioniert wie erwartet.	Bestanden
T-02	Modulübersicht prüfen	Unit-Test für ModulOverview ausführen. Geprüft werden Suchbegriff, Filterlogik, Kontextlabel und Fallback-Anzeigen.	Module werden im gewählten Kontext korrekt angezeigt. Such- und Filterfunktionen reduzieren die Liste korrekt.	Die Modulübersicht wurde erfolgreich getestet.	Bestanden
T-03	Handlungskompetenzbereich-Übersicht prüfen	Unit-Test für HkbOverview ausführen. Geprüft werden Suchlogik, HKB-Code, Titel, Beschreibung, Kontextlabel und Anzahl verknüpfter Handlungskompetenzen.	Handlungskompetenzbereiche werden korrekt gefiltert, formatiert und im Kontext dargestellt.	Die Übersicht funktioniert wie erwartet.	Bestanden

T-04	Handlungskompetenz-Übersicht prüfen	Unit-Test für HkOverview ausführen. Geprüft werden Suchlogik, HK-Code, Titel, Beschreibung, HKB-Zuordnung, Lehrjahranzeige und Modulanzahl.	Handlungskompetenzen werden korrekt angezeigt und gefiltert. Zugehörige Module werden korrekt gezählt.	Die Übersicht funktioniert korrekt.	Bestanden
T-05	Detailansichten prüfen	Unit-Tests für HkbDetail, HkDetail, ModulDetail und LernortDetail ausführen.	Detailansichten können erstellt werden und zentrale Zustände verursachen keine Testfehler.	Die Detailansicht-Tests wurden erfolgreich ausgeführt.	Bestanden
T-06	Selektoren prüfen	Unit-Tests für EFZ-, Fachrichtungs- und Lernortselektor ausführen.	Die Selektoren werden korrekt initialisiert und verursachen keine Template- oder Providerfehler	Alle Selektor-Tests wurden erfolgreich ausgeführt.	Bestanden
T-07	Filter-Sidebar prüfen	Unit-Test für FilterSidebar ausführen.	Die Sidebar wird korrekt erstellt und unterstützt die kontextabhängige Filterauswahl.	Der Test wurde erfolgreich ausgeführt.	Bestanden

T-08	Aktive Filteranzeige prüfen	Unit-Test für <code>ActiveFilterChips</code> ausführen.	Aktive Filter werden korrekt dargestellt und können nachvollziehbar entfernt werden.	Die aktive Filteranz eige wurde erfolgreich h getestet.	Besta nden
T-09	Suchseite und Suchresultate prüfen	Unit-Tests für <code>SearchPage</code> und <code>SearchResults</code> ausführen.	Suchseite und Resultatkomponen te werden korrekt erstellt. Leere oder vorhandene Resultate verursachen keine Fehler.	Beide Tests wurden erfolgreich h ausgefüh rt.	Besta nden
T-10	Gesamten automatisierten Testlauf prüfen	<code>npm test</code> ausführen und Terminalausgabe prüfen.	Alle Spec-Dateien laufen ohne Fehler durch.	17 Testdatei en und 19 Tests wurden bestande n.	Besta nden
T-11	Coverage prüfen	<code>npm run test:coverage</code> ausführen und Coverage-Bericht prüfen.	Die Coverage erreicht mindestens das gesetzte Ziel und zentrale Logik ist abgedeckt.	Die Coverage beträgt 82.22 % bei Statemen ts und Lines.	Besta nden
T-12	Produktionsbuild prüfen	<code>npm run build</code> ausführen.	Die Anwendung kann erfolgreich gebaut werden.	Die Bundle-Generier ung wurde	Besta nden

				erfolgreich h abgeschl ossen. Initial Total: 3.34 MB.	
T-13	Versionierung prüfen	git log -- oneline -- decorate -- graph -n 20 ausführen und Git- Historie prüfen.	Branches, Commits und Merges sind nachvollziehbar sichtbar.	Der Git- Log zeigt Feature- Branches , Merge- Commits und den aktuellen Main- Stand.	Besta nden

Auswertung der Testergebnisse

Die automatisierten Unit-Tests wurden erfolgreich abgeschlossen. Es wurden 17 Testdateien ausgeführt und alle 19 Tests bestanden. Dadurch konnte geprüft werden, dass zentrale Komponenten, Übersichten, Detailansichten, Selektoren, Filteranzeige, Suchseite und State-Logik ohne Fehler funktionieren.

Die Coverage-Auswertung zeigt eine Gesamt-Coverage von 82.22 % bei Statements und Lines. Besonders zentrale Bereiche wie die State-Verwaltung, Übersichtsseiten und einzelne Komponenten erreichten hohe Werte. Einige Detailansichten und UI-nahe Komponenten weisen tiefere Werte auf, weil nicht jede Template-Verzweigung und jeder mögliche Anzeigezustand vollständig automatisiert getestet wurde. Diese Bereiche wurden ergänzend manuell geprüft.

Der Produktionsbuild wurde erfolgreich ausgeführt. Damit wurde zusätzlich kontrolliert, dass die Anwendung nicht nur im Testlauf, sondern auch bei der Bundle-Erstellung technisch korrekt verarbeitet wird.

Die Git-Log-Screenshots dienen als Nachweis der Versionierung. Sie zeigen, dass die Umsetzung über Feature-Banches, Commits und Merge-Commits nachvollziehbar dokumentiert wurde.

Hinweise und Abgrenzung

Beim Testlauf erscheint der Hinweis, dass der Angular unit -test Builder experimentell ist. Dieser Hinweis betrifft das verwendete Tooling und stellt keinen Fehler in der Anwendung dar.

Da der Testlauf erfolgreich abgeschlossen wurde und alle Tests bestanden wurden, hatte dieser Hinweis keine negative Auswirkung auf das Testergebnis.

Nicht vollständig automatisiert getestet wurden alle möglichen Kombinationen aus API-Fehlern, leeren Datenbeständen und manuellen Benutzerinteraktionen. Diese Zustände wurden ergänzend manuell geprüft und in der Kontrollphase berücksichtigt.

Nachweise im Anhang

Nachweis	Beschreibung
Erfolgreicher Vitest-Testlauf im Terminal	Zeigt 17 bestandene Testdateien und 19 bestandene Tests.
Unit-Tests Coverage-Report 82.22 Prozent	Zeigt die erreichte Coverage von 82.22 % bei Statements und Lines.
Unit-Tests Bundle-Generation	Zeigt die erfolgreiche Bundle-Generierung und den erfolgreichen Build.
Git-Log im Terminal - Ausschnitt 1	Zeigt Branches, Commits und Merge-Historie als Versionierungsnachweis.
Git-Log im Terminal - Ausschnitt 2	Ergänzt den Git-Nachweis um weitere Commits und Branch-Informationen.